Programming assignment

2025 Fall - 機器學習

111652001　　吳文生

1. Use the **same code** from Assignment 2 - programming assignment 1 to calculate the error in approximating the derivative of the given function.
2. In this assignment, you will use a neural network to approximate both the **Runge function** and its **derivative**. Your task is to train a neural network that approximates:

   (a) The function $f(x)$ itself.

   (b) The derivative $f'(x)$

   You should define a **loss function** consisting of two components:

   (1) **Function loss:** the error between the predicted $f(x)$ and the true $f(x)$.

   (2) **Derivative loss:** the error between the predicted $f'(x)$ and the true $f'(x)$.

   Write a short report (1–2 pages) explaining method, results, and discussion including
   - Plot the true function and the neural network prediction together.
   - Show the training/validation loss curves.
   - Compute and report errors (MSE or max error).

Method

Our neural network consists of two hidden layers, each with 32 neurons, using the tanh activation function. The output layer is a single linear neuron. The network is designed to simultaneously approximate the Runge function

$$f(x) = \frac{1}{1+25x^2},\ x \in [-1,\ 1]$$

and its derivative

$$f'(x) = \frac{-50x}{(1+25x^2)^2},\ x \in (-1,1).$$

The training set contains 1200 uniformly sampled input points from the interval $[-1,1]$, along with corresponding function values and true derivatives computed analytically. The validation set consists of 300 points sampled similarly. The model was trained for 100 epochs. The loss function used is a composite of two mean squared error (MSE) components: the error between predicted and true function values, and the error between predicted and true derivatives. Training and validation losses were recorded every epoch for performance evaluation.

Results

1. The neural network prediction closely matches the true Runge function. Equally, the predicted derivative aligns well with the true analytical derivative. Both sets of curves nearly overlap without significant deviation, indicating high fidelity in approximation.

2. Training and validation loss curves demonstrate rapid convergence and stabilization. The behavior confirms effective learning and good generalization of the network for both function and derivative approximations.

3. Quantitative error metrics calculated on the validation set show:
    - Function approximation: MSE approximately 0.000003, maximum absolute error about 0.00250.
    - Derivative approximation: MSE approximately 0.000023, maximum absolute error about 0.02162.

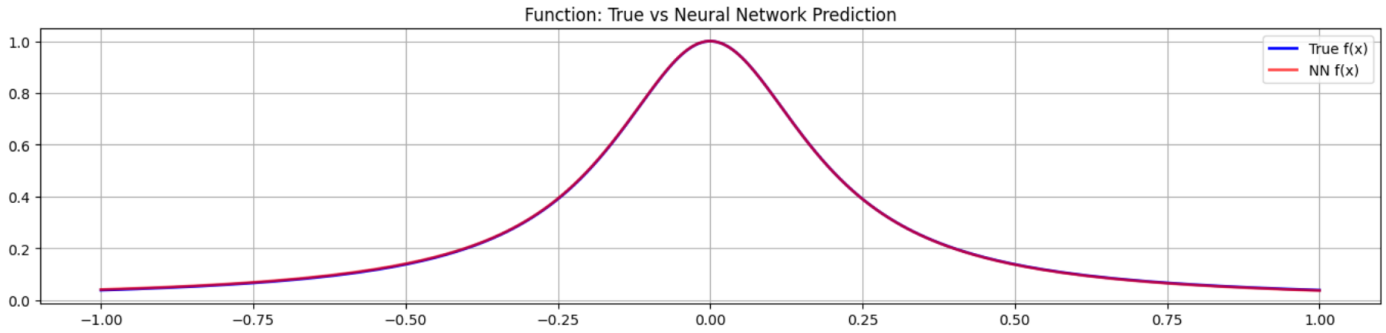

Figure 1 Function: True vs Neural Network Prediction

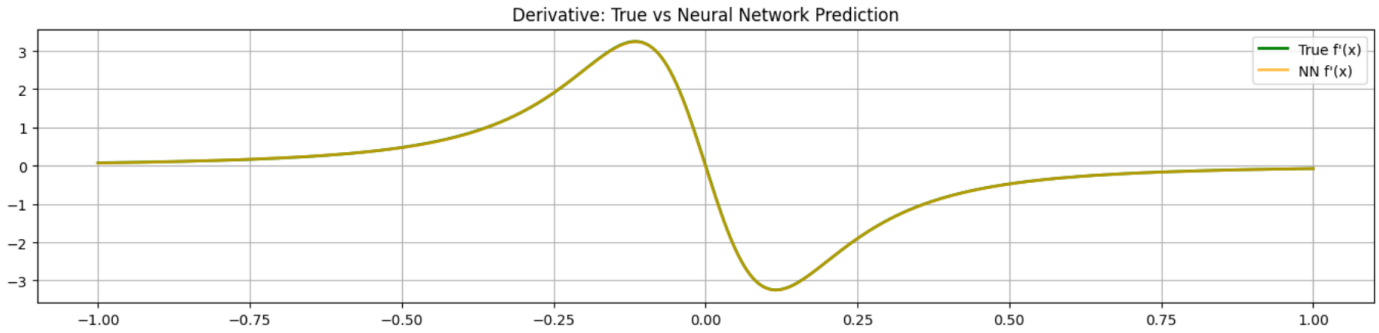

Figure 2 Derivative: True vs Neural Network Prediction

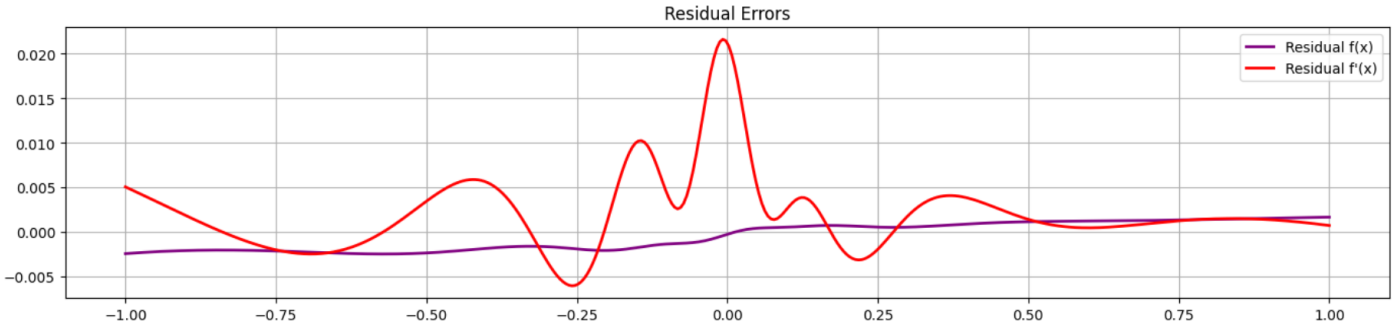

Figure 3 Residual Errors

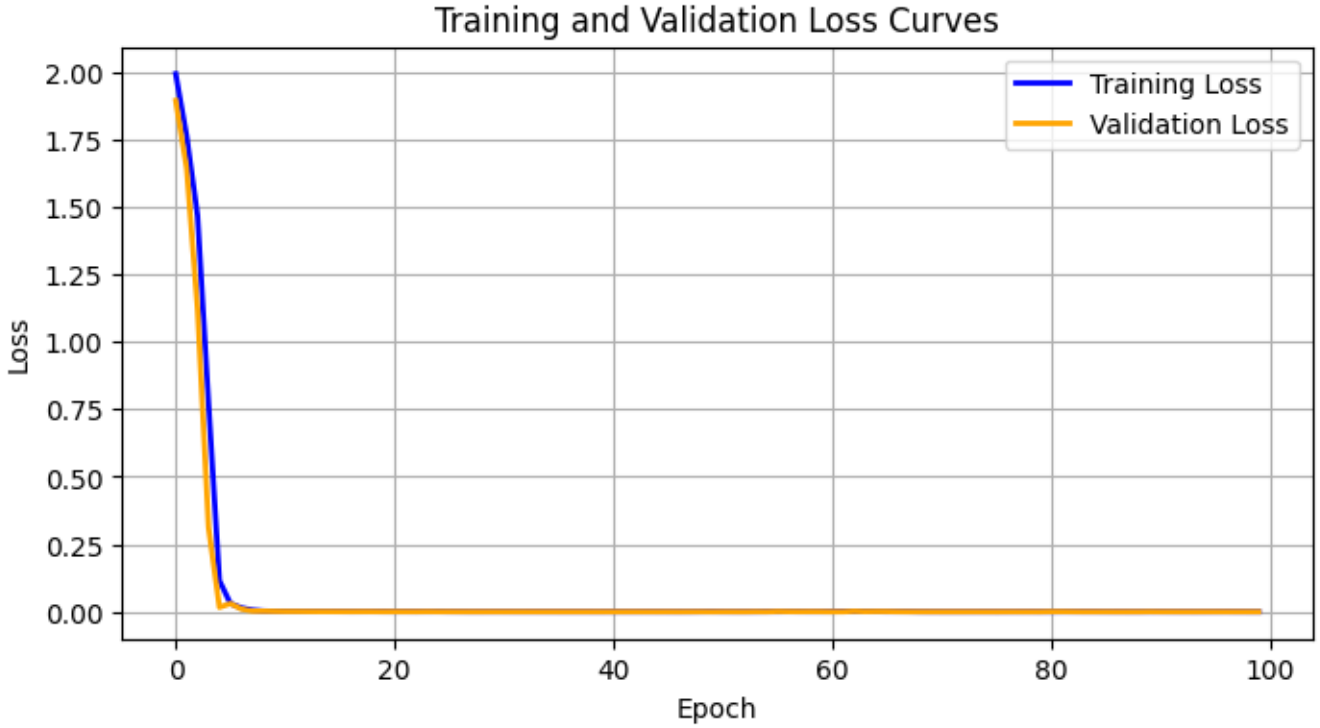

Figure 4 Training and Validation Loss

```
MSE f(x): 0.000003, Max Error f(x): 0.00250
MSE f'(x): 0.000023, Max Error f'(x): 0.02162
```

Figure 5 MSE and max error

Code

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt

# Data Preparation
def runge_function(x):
    return 1.0 / (1 + 25 * x**2)

def runge_derivative(x):
    return -50 * x / (1 + 25 * x**2)**2

# Sample points
np.random.seed(0)
x_train = np.linspace(-1, 1, 1200)
y_train = runge_function(x_train)
dy_train = runge_derivative(x_train)
x_val = np.linspace(-1, 1, 300)
y_val = runge_function(x_val)
dy_val = runge_derivative(x_val)

# PyTorch tensors
x_train_tensor = torch.FloatTensor(x_train).view(-1, 1)
y_train_tensor = torch.FloatTensor(y_train).view(-1, 1)
dy_train_tensor = torch.FloatTensor(dy_train).view(-1, 1)
x_val_tensor = torch.FloatTensor(x_val).view(-1, 1)
y_val_tensor = torch.FloatTensor(y_val).view(-1, 1)
dy_val_tensor = torch.FloatTensor(dy_val).view(-1, 1)

train_ds = TensorDataset(x_train_tensor, y_train_tensor, dy_train_tensor)
val_ds = TensorDataset(x_val_tensor, y_val_tensor, dy_val_tensor)
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=128, shuffle=False)

# Model definition (tanh activation)
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(1, 32)
        self.fc2 = nn.Linear(32, 32)
        self.fc3 = nn.Linear(32, 1)
    def forward(self, x):
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
optimizer = torch.optim.Adam(net.parameters(), lr=0.01)
criterion = nn.MSELoss()

# Training loop
num_epochs = 100
train_losses, val_losses = [], []

for epoch in range(num_epochs):
```

```
    net.train()
    batch_train_losses = []
    for xb, yb, dyb in train_loader:
        xb_ = xb.clone().detach().requires_grad_(True)
        optimizer.zero_grad()
        out = net(xb_)
        loss_f = criterion(out, yb)
        grad_out = torch.autograd.grad(
            outputs=out,
            inputs=xb_,
            grad_outputs=torch.ones_like(out),
            create_graph=True,
            retain_graph=True,
            only_inputs=True
        )[0]
        loss_df = criterion(grad_out, dyb)
        loss = loss_f + loss_df
        loss.backward()
        optimizer.step()
        batch_train_losses.append(loss.item())
    train_losses.append(np.mean(batch_train_losses))

    net.eval()
    with torch.enable_grad():
        x_val_tensor_ = x_val_tensor.clone().detach().requires_grad_(True)
        out_val = net(x_val_tensor_)
        val_loss_f = criterion(out_val, y_val_tensor)
        grad_val = torch.autograd.grad(
            outputs=out_val,
            inputs=x_val_tensor_,
            grad_outputs=torch.ones_like(out_val),
            create_graph=False,
            retain_graph=False,
            only_inputs=True
        )[0]
        val_loss_df = criterion(grad_val, dy_val_tensor)
        val_loss = val_loss_f + val_loss_df
        val_losses.append(val_loss.item())


x_plot = np.linspace(-1, 1, 400)
y_true = runge_function(x_plot)
dy_true = runge_derivative(x_plot)

x_plot_tensor = torch.FloatTensor(x_plot).view(-1,1).requires_grad_(True)
y_pred_tensor = net(x_plot_tensor)
y_pred = y_pred_tensor.detach().numpy()

dy_pred = torch.autograd.grad(
    outputs=y_pred_tensor,
    inputs=x_plot_tensor,
    grad_outputs=torch.ones_like(y_pred_tensor),
    create_graph=False
)[0].detach().numpy()

# Plotting results
plt.figure(figsize=(14, 10))

plt.subplot(3, 1, 1)
plt.plot(x_plot, y_true, label="True f(x)", color='blue', linewidth=2)
plt.plot(x_plot, y_pred, label="NN f(x)", color='red', linewidth=2, alpha=0.7)
```

```
plt.legend()
plt.title("Function: True vs Neural Network Prediction")
plt.grid(True)

plt.subplot(3, 1, 2)
plt.plot(x_plot, dy_true, label="True f'(x)", color='green', linewidth=2)
plt.plot(x_plot, dy_pred, label="NN f'(x)", color='orange', linewidth=2, alpha=0.7)
plt.legend()
plt.title("Derivative: True vs Neural Network Prediction")
plt.grid(True)

plt.subplot(3, 1, 3)
plt.plot(x_plot, y_true - y_pred.flatten(), label="Residual f(x)", color='purple', linewidth=2)
plt.plot(x_plot, dy_true - dy_pred.flatten(), label="Residual f'(x)", color='red', linewidth=2)
plt.legend()
plt.title("Residual Errors")
plt.grid(True)

plt.tight_layout()
plt.show()

# Error computation
mse_f = np.mean((y_true - y_pred.flatten())**2)
max_error_f = np.max(np.abs(y_true - y_pred.flatten()))
mse_df = np.mean((dy_true - dy_pred.flatten())**2)
max_error_df = np.max(np.abs(dy_true - dy_pred.flatten()))

print(f"MSE f(x): {mse_f:.6f}, Max Error f(x): {max_error_f:.5f}")
print(f"MSE f'(x): {mse_df:.6f}, Max Error f'(x): {max_error_df:.5f}")

plt.figure(figsize=(8, 4))
plt.plot(train_losses, label="Training Loss", color='blue', linewidth=2)
plt.plot(val_losses, label="Validation Loss", color='orange', linewaidth=2)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss Curves")
plt.legend()
plt.grid(True)
plt.show()
```